

Как писать логи правильно в Spring Boot + Lombok?

О нас

FAANG School – IT онлайн-школа с первоклассными образовательными продуктами, которая выпускает разработчиков с гибким мышлением и мощным стеком.

Наша миссия

Помогаем реализовать амбиции, стать профессиональными разработчиками и получить работу мечты в лучших IT-компаниях в кратчайшие сроки!

Что такое Java Буткемп?

Java Буткемп — это интенсивное обучение в формате командной работы над мощным проектом, где ты получишь максимально приближенный к реальному опыт для успешного трудоустройства с высокой зарплатой в кратчайшие сроки.

Для этого мы адаптировали практики разработки BigTech-компаний, внедрили передовые инструменты индустрии и организовали работу по гибкой методологии Scrum, чтобы ты выделялся среди других кандидатов на рынке.

**Стремишься стать востребованным разработчиком и получить работу мечты в кратчайшие сроки?
Скорее записывайся на Java Буткемп!**

[**ЗАПИСАТЬСЯ НА ЭКСКУРСИЮ**](#)

Логирование — это одна из тех тем, которые на первый взгляд кажутся тривиальными, но при глубоком изучении становятся сложными и многогранными. Люди создают фреймворки для логирования, спорят о лучших практиках, документируют соглашения по логам... Всё это может удивить начинающего разработчика. Этот гайд даст вам базовое понимание основных концепций и рекомендаций по лучшим практикам логирования.

Когда вы впервые вывели на терминал строку "Hello, World!", вы уже занимались логированием. Печать логов — это просто и удобно для понимания кода или отладки небольших программ. Однако для сложных приложений существуют специальные библиотеки логирования, которые позволяют управлять тем, что и когда записывается в логи. Можно регулировать объем логов с помощью уровней логирования и контролировать формат логов. Фреймворки также добавляют в логи контекстную информацию — имена потоков, хосты, идентификаторы, — что помогает при отладке. Также они отлично работают с системами управления логами, которые собирают сообщения в единое место, позволяя фильтровать и искать нужные записи.

Использование уровней логирования

Фреймворки логирования поддерживают уровни логов, которые позволяют фильтровать сообщения по их важности. Когда вы устанавливаете уровень логирования, все сообщения на этом уровне или выше будут записаны, а менее важные будут проигнорированы. Уровни обычно настраиваются как глобально, так и для отдельных пакетов или классов. Это позволяет регулировать объем логов в зависимости от ситуации.

Пример: фрагмент Java-конфигурации `log4j.properties`, который задает уровень логирования `ERROR` для всего приложения и уровень `INFO` для пакета `com.foo.bar`:

```
# установить уровень ERROR для корневого логгера и запись в файл через FileAppender
log4j.rootLogger=ERROR,fout
# установить уровень INFO для пакета com.foo.bar
log4j.logger.com.foo.bar=INFO
```

Для того чтобы уровни логов были полезны, необходимо правильно использовать критичность каждого сообщения. Хотя стандарты уровней логирования могут немного различаться, обычно выделяют следующие уровни:

- **TRACE**

Это самый детализированный уровень, который включается только для отдельных пакетов или классов. Обычно он используется только на этапе разработки. Если вам нужны строчные логи или дампы структур данных, этот уровень подойдет. Если вы часто используете `TRACE`, подумайте об использовании отладчика для пошагового выполнения кода.

- **DEBUG**

Используется, когда сообщение полезно при решении проблемы в продакшене, но не нужно в обычных условиях. Не злоупотребляйте этим уровнем, иначе отладка станет сложной из-за обилия сообщений. Для этого лучше подходит TRACE.

- **INFO**

Это полезная информация о состоянии приложения, которая не указывает на наличие проблем. Сообщения о состоянии системы, такие как «Сервис запущен» или «Ожидание на порту 5050», относятся сюда. INFO — это уровень логирования по умолчанию. Не добавляйте слишком много бесполезных логов на этот уровень — для «на всякий случай» больше подходит TRACE или DEBUG. Логи уровня INFO должны быть информативными в нормальной эксплуатации.

- **WARN**

Эти сообщения указывают на потенциально проблемные ситуации. Если при записи предупреждения нет явного действия, которое нужно предпринять, лучше использовать INFO.

- **ERROR**

Это ошибки, требующие внимания. Например, невозможность записать данные в базу данных должна логироваться как ERROR. Сообщения уровня ERROR должны быть достаточно подробными для диагностики проблемы: включайте стек трейс и информацию о том, какие действия предпринимает программа.

- **FATAL**

Это сообщения о критических ошибках. Если программа сталкивается с проблемой, из-за которой она должна немедленно завершить работу, логируйте причину на уровне FATAL. Включайте информацию о состоянии программы и данные для восстановления или диагностики.

Пример Java-класса с использованием Log4j:

```
import org.apache.log4j.Logger;

public class Example {
    // Инициализация логгера
    private static final Logger logger = Logger.getLogger(Example.class);

    public static void main(String[] args) {
        // Логирование сообщения уровня INFO
        logger.info("Это информационное сообщение");
    }
}
```


Делайте логи атомарными

Если информация полезна только в сочетании с другими данными, записывайте её атомарно в одно сообщение. Атомарные логи, содержащие всю необходимую информацию в одной строке, лучше работают с системами агрегации логов.

Не рассчитывайте на то, что логи будут просматриваться в определённом порядке — многие операционные инструменты могут переставлять сообщения или даже терять их.

Не полагайтесь на метки времени системных часов для упорядочивания сообщений: системные часы могут сбрасываться или расходиться между разными компьютерами, на которых запущены копии вашего приложения.

Избегайте символов новой строки в логах, так как многие системы обработки логов воспринимают новую строку как отдельное сообщение.

Убедитесь, что стектрейсы записываются в одно сообщение, поскольку при печати они часто включают в себя переносы строк.

Если лог не может быть выведен атомарно, добавляйте уникальный идентификатор в сообщения, чтобы их можно было сопоставить позже.

Следите за производительностью

Чрезмерное логирование снижает производительность. Логи необходимо где-то сохранять — на диск, в консоль или в удалённую систему. Строки должны быть объединены и отформатированы перед записью. Используйте параметризованные сообщения и асинхронные аппендеры для ускорения логирования.

Конкатенация строк медленная и может серьёзно снизить скорость выполнения в производительно-важных операциях. Когда строка передаётся в метод логирования, её конкатенация выполняется независимо от уровня логирования, так как аргументы оцениваются до того, как они передаются в метод.

Фреймворки логирования предлагают механизмы, позволяющие отложить конкатенацию строк до того момента, когда она действительно понадобится. Некоторые фреймворки заставляют лог-сообщения попадать в замыкания, которые не вычисляются, если строка не будет записана, в то время как другие поддерживают параметризованные сообщения.

Проверь свои знания в Java!

Пройди тест и узнай, как с твоим уровнем в Java стать востребованным программистом.
Бонус — Roadmap по Java

[ПРОЙТИ ТЕСТ](#)



Вот пример на Java, который демонстрирует влияние на производительность при использовании конкатенации строк и правильное использование параметризованных сообщений в логировании:

```
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

public class Example {

    private static final Logger logger = LogManager.getLogger(Example.class);

    public static void main(String[] args) {
        String message = "example message";
        int someValue = 42;

        // Пример с конкатенацией строк - это медленно и выполняется всегда, даже если уровень
        // логирования ниже.
        logger.trace("Trace log with concatenation: " + message + ", value: " + someValue);

        // Параметризованное сообщение - вычисляется только если trace включен, что быстрее.
        logger.trace("Trace log with parameterized message: {}, value: {}", message, someValue);

        // Более сложный пример с циклом, где каждая конкатенация строки замедляет
        // производительность.
        for (int i = 0; i < 1000; i++) {
            String detailMessage = "Detailed message " + i;
            // Конкатенация строк перед вызовом метода логирования
            logger.debug("Processing: " + detailMessage); // Это медленно, т.к. строка создается всегда.

            // Параметризованный вызов - строка создается только при активном уровне DEBUG
            logger.debug("Processing message: {}", detailMessage); // Это быстрее.
        }
    }
}
```

Также можно управлять производительностью с помощью аппендеров. Аппендеры направляют логи в разные места: консоль, файл или удалённую систему агрегации логов.

По умолчанию аппендеры работают в потоке вызывающего кода, как обычный вызов метода `print`. Асинхронные аппендеры записывают сообщения в лог, не блокируя выполнение потоков. Это повышает производительность, так как приложению не нужно ждать записи логов.

Аппендеры с буферизацией накапливают сообщения в памяти перед записью на диск, что улучшает скорость записи. Кэш страниц операционной системы также помогает буферизировать логи для повышения производительности.

Однако асинхронная запись и буферизация могут привести к потере логов, если приложение аварийно завершится, так как не все сообщения гарантированно будут записаны на диск.

Имейте в виду, что изменение уровня логирования и конфигурации может устранить состояния гонки и баги, так как замедляет выполнение приложения. Если вы включили подробное логирование для отладки и обнаружили, что баг исчез, возможно, причиной стал сам факт включения логов.

Не логируйте конфиденциальные данные

Будьте осторожны с конфиденциальной информацией. Лог-сообщения не должны содержать такие данные, как пароли, токены безопасности, номера кредитных карт или электронные адреса. Это может показаться очевидным, но легко допустить ошибку — например, просто залогировав URL или HTTP-ответ, вы можете случайно передать информацию, которую системы агрегации логов не защищают.

Большинство фреймворков поддерживают правила замены и редактирования строк, настроив которые можно обезопасить данные, но не полагайтесь на них как на единственную защиту. Будьте немного параноиком:) Ведь логирование конфиденциальной информации может создать угрозы безопасности и нарушить требования по защите данных.

@Log и @Slf4j в Spring Boot-приложениях

Spring Boot стал популярным выбором для разработки приложений благодаря своей простоте, мощным возможностям и надежной экосистеме. Один аспект, который часто упускается, но является крайне важным для любого приложения, особенно в продакшене, — это логирование. Spring Boot предоставляет набор встроенных инструментов для логирования, а с помощью библиотеки Lombok разработчики могут еще больше упростить логирование, используя аннотации, такие как `@Log` и `@Slf4j`.

Основы логирования в Spring Boot

При работе с Spring Boot разработчики получают мощную и настраиваемую систему логирования «из коробки».

По умолчанию Spring Boot включает в себя Simple Logging Facade for Java (SLF4J), работающий в связке с Logback. SLF4J выступает в роли абстракции, что позволяет подставлять любой предпочитаемый фреймворк логирования на этапе развертывания приложения, в то время как Logback служит реализацией по умолчанию. Это сочетание дает разработчикам гибкую и высоконастраиваемую систему логирования.

Настройка логирования в файле `application.properties`

Spring Boot позволяет разработчикам настраивать систему логирования с помощью файла `application.properties` (или `application.yml`). Вот несколько часто используемых конфигураций:

1. Установка глобального уровня логирования:

Чтобы задать базовый уровень для всех логгеров:

```
logging.level.root=WARN
```

2. Установка уровня логирования для конкретного пакета или класса:

```
logging.level.org.springframework.web=DEBUG
logging.level.com.myapp.service=INFO
```


3. Вывод логов в файл:

По умолчанию логи выводятся в консоль.
Если вы хотите сохранять их в файл:

```
logging.file.name=myapp.log
```

4. Ротация логов:

Для крупных приложений логи могут быстро увеличиваться в объеме. Чтобы управлять их размером, Spring Boot поддерживает ротацию логов:

```
logging.file.max-size=10MB  
logging.file.max-history=10
```

Кастомизация формата логов

По умолчанию вывод логов в Spring Boot краткий и удобный для разработчиков. Однако для специфических требований можно настроить формат вывода. С помощью свойств `logging.pattern.console` и `logging.pattern.file` вы можете задать пользовательские шаблоны для вывода в консоль и в файл соответственно.

Пример настройки шаблона для консоли:

```
logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss} - %logger{36} - %msg%n
```

Этот шаблон включает метку времени, имя логгера (до 36 символов) и само сообщение.

Введение в Lombok и аннотации для логирования

Project Lombok — это настоящее спасение для Java-разработчиков, когда речь идет о сокращении рутинного кода. Java известен своей многословностью, что часто полезно для понимания логики программы, но может становиться обременительным при выполнении рутинных задач, таких как настройка логирования. Здесь и приходит на помощь Lombok, упрощая процесс логирования.

Lombok — это процессор аннотаций, работающий на этапе компиляции. Вместо того чтобы писать повторяющийся код вручную или полагаться на вашу среду разработки для его генерации, Lombok предоставляет аннотации, которые указывают компилятору автоматически генерировать код. Это не только делает кодовую базу чище, но и ускоряет процесс разработки, снижая вероятность ошибок.

Пет-проект который принесет тебе работу

О том, как избежать типичных ошибок новичков и улучшить свой пет-проект, главный преподаватель FAANG School — Влад Мишустин рассказал в этом видео!

[СМОТРЕТЬ](#)



Аннотации для логирования в Lombok

Lombok предлагает множество аннотаций для разных задач, таких как `@Data` для автоматической генерации методов `getter`, `setter` и других общих методов. Однако здесь мы сосредоточимся на аннотациях для логирования:

- **@Slf4j:** Это самая распространенная аннотация для логирования в Spring Boot приложениях. При её применении к классу автоматически создается статический логгер SLF4J с именем `log`, который использует фасад SLF4J для логирования.
- **@Log:** Эта аннотация используется для приложений, которые полагаются на фреймворк логирования `java.util.logging`. Подобно `@Slf4j`, она автоматически создает статический логгер с именем `log`.

Интеграция Lombok с Spring Boot

Для использования Lombok в проекте Spring Boot, просто добавьте зависимость Lombok в файл сборки проекта.

Для Maven:

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>CheckLatestVersion</version>
  <scope>provided</scope>
</dependency>
```

Для Gradle:

```
compileOnly 'org.projectlombok:lombok:CheckLatestVersion'
annotationProcessor 'org.projectlombok:lombok:CheckLatestVersion'
```

Примечание: Убедитесь, что используете последнюю версию Lombok при добавлении зависимости.

Если используете IntelliJ IDEA, то не забудьте также поставить в нее плагин Lombok, иначе аннотации в ней не отработают.

Происхождение и используемые фреймворки

- @Slf4j:**
- **Происхождение:** SLF4J расшифровывается как Simple Logging Facade for Java.
 - **Целевой фреймворк:** Аннотация создана для работы с фасадом SLF4J. SLF4J представляет собой простую абстракцию для различных систем логирования. Использование `@Slf4j` обеспечивает гибкость, если в будущем потребуется перейти с Logback (логгер по умолчанию в Spring Boot) на другой фреймворк, например, Log4j2.

- @Log:**
- **Происхождение:** Эта аннотация относится к пакету `java.util.logging` (часто сокращаемому как JUL).
 - **Целевой фреймворк:** `@Log` ориентирован на встроенную систему логирования Java — `java.util.logging`. Хотя она менее популярна в экосистеме Spring, JUL иногда используется в устаревших или специфических приложениях, где встроенный механизм логирования Java является преимуществом.

Сценарии использования

- @Slf4j:**
- **Современные Spring Boot приложения:** Так как Spring Boot по умолчанию использует SLF4J и Logback, аннотация `@Slf4j` является предпочтительным выбором для большинства проектов Spring Boot.
 - **Необходимость в совместимости:** Если предполагается необходимость переключения между различными фреймворками для логирования, фасад SLF4J с поддержкой `@Slf4j` обеспечит гладкий переход.
- @Log:**
- **Устаревшие приложения:** В старых Java-приложениях, где уже глубоко внедрена система `java.util.logging`, аннотация `@Log` может добавить удобство в написании современного кода.
 - **Проекты с минимальными зависимостями:** Для проектов, где важно минимизировать внешние зависимости, использование встроенного механизма логирования Java с помощью `@Log` может быть полезным.

Как использовать

Для обеих аннотаций реализация предельно проста. После интеграции Lombok в проект, просто добавьте аннотацию к классу.

- **Пример с `@Slf4j`:**

```
import lombok.extern.slf4j.Slf4j;

@Slf4j
public class OrderService {
    public void placeOrder() {
        log.info("Заказ успешно размещён с использованием SLF4J");
    }
}
```

- **Пример с `@Log`:**

```
import java.util.logging.Logger;
import lombok.extern.java.Log;

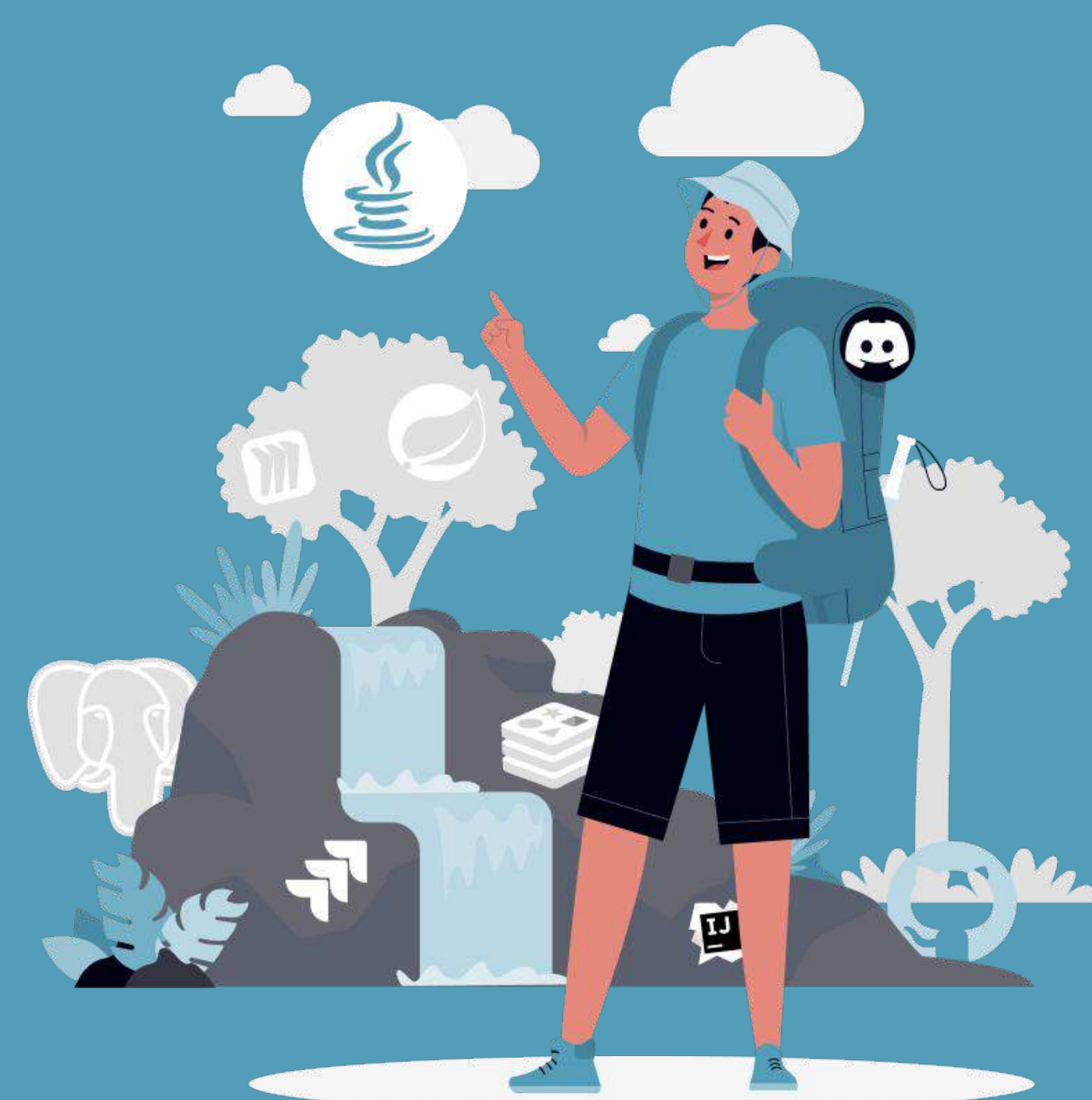
@Log
public class InventoryChecker {
    public void checkStock() {
        log.info("Проверка запасов выполнена с использованием java.util.logging");
    }
}
```


В обоих случаях Lombok автоматически создаст статический экземпляр логгера с именем `log`, что избавит вас от необходимости вручную создавать и инициализировать логгер в каждом классе.

Почувствуй себя студентом FAANG School

Запишись на интерактивную экскурсию по Java Буткемпу!

[ЗАПИСАТЬСЯ НА ЭКСКУРСИЮ](#)



Лучшие практики

Эффективное логирование — это не только правильный выбор инструментов, но и соблюдение лучших практик. Хотя аннотация `@Slf4j` устраняет шаблонный код и упрощает создание логгеров, важно следовать ряду рекомендаций, чтобы логирование приносило максимальную пользу.

1. Логируйте содержательные сообщения

Каждое сообщение в логах должно быть информативным и понятным даже для тех, кто не знаком с кодом. Избегайте неясных сообщений, таких как «Произошла ошибка».

Пример:

```
@Slf4j
public class PaymentService {
    public void processPayment(Payment payment) {
        if (payment == null) {
            log.error("Не удалось обработать платеж: объект Payment равен null.");
        }
        // ...
    }
}
```

2. Используйте правильные уровни логирования

Неправильное использование уровней логирования может привести либо к потере важной информации, либо к «раздуванию» логов. Важно выбирать правильный уровень для каждого случая:

- **ERROR:** Для серьёзных ошибок, которые могут прервать работу приложения.
- **WARN:** Для потенциальных проблем, которые не останавливают выполнение, но требуют внимания.

- **INFO:** Общие сообщения о состоянии приложения.
- **DEBUG:** Полезные для отладки, но слишком подробные для обычных логов.
- **TRACE:** Очень детализированные сообщения для глубокой отладки.

3. Избегайте логирования конфиденциальной информации

Никогда не логируйте чувствительные данные, такие как пароли, номера кредитных карт или персональные данные (PII). Это не только хорошая практика безопасности, но и в некоторых юрисдикциях является юридическим требованием.

4. Используйте параметризованное логирование

Вместо конкатенации строк используйте параметризованные сообщения, которые предоставляет SLF4J. Это повышает производительность и предотвращает ненужное создание строк.

Пример:

```
String orderId = "012345";  
log.info("Обработка заказа с ID: {}", orderId);
```

5. Корректно обрабатывайте исключения

При логировании исключений важно фиксировать весь стек вызовов (stack trace), чтобы можно было диагностировать причину ошибки. SLF4J упрощает этот процесс:

```
try {  
    // код, который может вызвать исключение  
} catch (Exception ex) {  
    log.error("Произошла ошибка при обработке", ex);  
}
```

6. Не полагайтесь только на логи для мониторинга

Логи незаменимы для диагностики, но они не должны быть единственным средством мониторинга. Используйте метрики, уведомления и другие инструменты мониторинга в дополнение к логам.

7. Настройте ротацию и архивирование логов

Убедитесь, что ваша система логирования настроена на ротацию логов, чтобы файлы не становились слишком большими, а старые логи архивировались для возможного анализа. Это можно сделать с помощью параметров `logging.file.max-size` и `logging.file.max-history` в Spring Boot.

8. Избегайте логирования внутри циклов

Логирование внутри циклов, особенно плотных, может значительно замедлить работу приложения и сгенерировать огромное количество логов. Будьте осторожны при логировании на уровнях DEBUG или TRACE в таких местах.

9. Поддерживайте единообразие

Соблюдайте единый стиль и паттерны логирования по всему приложению. Это улучшает читаемость и позволяет инструментам автоматической обработки логов легко анализировать сообщения.

10. Регулярно пересматривайте и очищайте логи

Со временем в логах может накапливаться «шум» — сообщения, которые когда-то были полезными, но теперь только засоряют журнал. Регулярно пересматривайте и очищайте такие сообщения, чтобы логи оставались полезным инструментом.

Заключение

Логирование — важнейший аспект работы любого приложения. Хотя Spring Boot предоставляет мощную систему логирования «из коробки», использование аннотаций Lombok, таких как `@Log` и `@Slf4j`, может упростить и улучшить этот процесс. Эти инструменты помогают сократить шаблонный код и предоставляют интуитивный способ добавления сообщений в логи, позволяя разработчикам сосредоточиться на создании функциональных и эффективных приложений. Как и с любыми инструментами, важно использовать их эффективно и ответственно, следуя лучшим практикам, чтобы ваши логи всегда приносили максимальную пользу.



Хочешь узнать больше о том,
как именно тебе начать
карьеру в IT в 2024?

Переходи по ссылке и получай
БЕСПЛАТНУЮ карьерную
консультацию от команды
FAANG School!

[ЗАПИСАТЬСЯ](#)

Подписывайся на нас
в соц.сетях:

 [Telegram](#)

 [YouTube](#)

 [ВК](#)

